

Section 4: Architectural Patterns & Styles

Event-Driven Architecture

In this session, we will explore how event-driven systems enable asynchronous and decoupled communication in modern architectures. We'll cover key components like PubSub model and event streaming. We'll look at the major components such as event producers, brokers, and consumers, and we will try to see how event-driven architecture is helpful in modern architecture.

By the end of this lecture, you will understand how event-driven architecture improves scalability, fault tolerance, and flexibility in large-scale distributed systems.

What is Event-Driven Architecture?

Event-driven architecture is a system design approach where different components communicate through events instead of making direct calls to each other. This allows the system to be more flexible and scalable.

The below are some of the key characteristics of event-driven architecture:

- **Asynchronous Processing:** Event-driven architecture relies on asynchronous processing. Unlike traditional request-response models where services wait for a response, event-driven architecture allows services to continue working while events are processed in the background.
- **Loose Coupling:** They promote loose coupling as the components communicate through events rather than direct communications, they can evolve independently without breaking the system.
- **Scalability & Flexibility:** Since event-driven systems don't have tight dependencies, they can handle spikes in traffic much more efficiently.

The below are some of the main reasons why event-driven architectures are used to build modern applications:

- It improves the system responsiveness by enabling real-time event processing, making it ideal for applications like financial transactions, notifications, and monitoring systems.
- It also supports complex workflows where multiple services need to react to a change, such as order processing systems, IoT data handling, or supply chain management.

In summary, event-driven architecture is a powerful approach that allows systems to be more dynamic, resilient, and scalable.

Synchronous Vs. Asynchronous Systems

Synchronous System: In a synchronous system, a request is sent and the sender waits for a response before proceeding. This means the process is blocked, nothing else can happen until a response is received. This approach creates a tight coupling between components, as one service depends on another to function properly. A common example of synchronous communication is a traditional HTTP call. When a client sends a request, it waits for a response before moving forward. While this model is simple and easy to understand, it can lead to bottlenecks, especially when multiple services rely on each other.

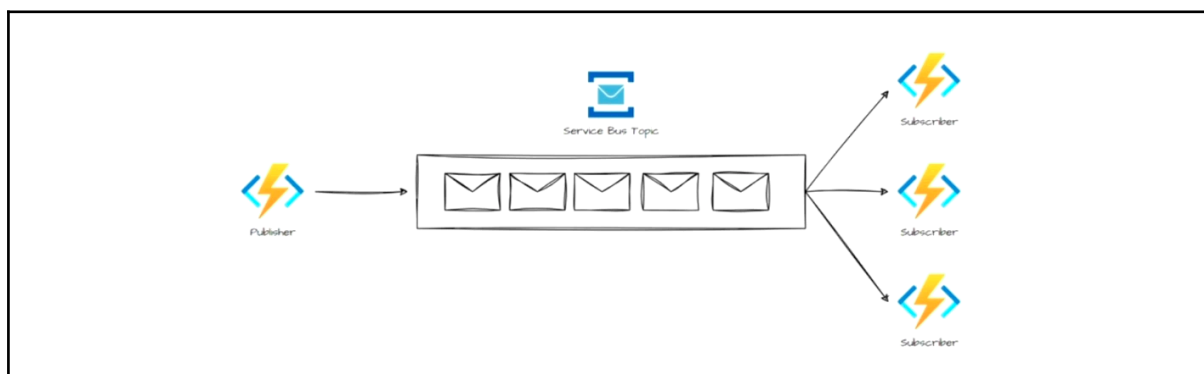
Asynchronous System: In contrast to synchronous, asynchronous communication operates on a non-blocking model. A service can send an event and continue processing other tasks without waiting for an immediate response. This enables loose coupling, meaning components are independent and involved separately, making the system much more flexible and scalable. Examples of asynchronous communication are message queues and event brokers like kafka and Revit MQ. These systems allow different components to react to events whenever they are ready.

So in summary, synchronous systems are direct and simple, but they can become bottlenecks. Whereas asynchronous systems promote stability and resilience by decoupling components. This is why event-driven architecture is so powerful.

Pub-Sub vs. Event Streaming

There are 2 fundamental event-driven messaging patterns :

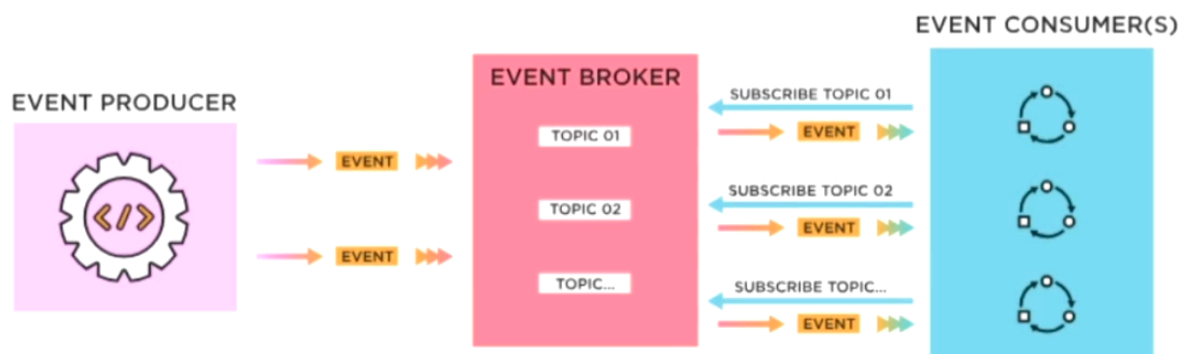
Publish Subscribe: In a publish-subscribe model, events are broadcasted to multiple subscribers. The publisher doesn't need to know who the subscribers are. It simply sends an event and any interested subscriber will receive it. Each subscriber gets the event once, meaning the message is not stored for later retrieval. In a real-world example of publisher-subscriber, like in a notification system, when a user posts an update, multiple followers get notified instantly. Nowadays technologies like RevitMQ and AWS simple notification service actually implements this model.



Event Streaming: Unlike pub-sub, event streaming is designed for long-term event retention. Events are stored in an ordered fashion, allowing consumers to process them at different times. This means consumers don't have to be online when an event is published. They can receive the past events from the log and process them at their own pace. A common use case for event streaming is processing real-time analytics, such as tracking a website visit over a time. We have popular technologies for event streaming like Apache Kafka and AWS Kinesis.

So to summarize, pubsub is great for real-time notifications, while event streaming is better suited for scenarios where event history and ordered processing matter. Choosing the right approach depends on your use case of the application.

Key Components of an Event-Driven System



The key components that work together to form an event-driven system are:

- **Event Producers:** These are responsible for generating events. They are the starting point of an event-driven system. These are the components that generate events based on system activity. Producers can be anything from using actions like clicking a button, microservices updating a database, or IoT devices sending censored data. For example, in an e-commerce system, when the user places an order, an order-placed event is generated by the order service.
- **Event Brokers:** Event brokers are responsible for transmitting and storing events. They act as intermediaries between the event producers and consumers. They ensure events are reliably transmitted and, in some cases, stored for later consumption. They help in decoupling components, making the system more scalable and fault-tolerant. Common event brokers include Apache Kafka, RabbitMQ, and AWS EventBridge. Example: In a real-time analytics system, kafka can store a stream of events for multiple sources and distribute them to multiple consumers.

- **Event Consumers:** These are the components that are reacting to events. Event consumers are the components that listen for events and take actions based on them. Consumers can be anything from database updater, a notification service, or a fraud detection system. Example: A ride-sharing app, when a user requests a ride, a consumer service matches the user with the nearby driver.
- **Event Storage** (Not visible in diagram but is important): It's responsibility is persisting events for replay. It's not present in all systems, but in many cases, we don't just want to process the events in real-time, but we also need to store them for historical analysis, debugging, or replaying in case of failures. In these components, log-based storage allows events to be replayed and reprocessed, which is especially useful in systems like financial transactions or audit logs. Examples like Kafka and AWS Kinesis are very helpful in event storage.

So to summarize, together these 4 components form the backbone of an event-driven system, enabling scalability, resilience and loosely coupled architecture.

Challenges in Event-Driven Architecture

Event-driven architecture brings a lot of benefits, but it also comes with its own set of challenges. Some of the key challenges and the way to handle them are given below:

- **Eventual Consistency:** Unlike traditional request-response systems, where updates happen synchronously, event-driven systems rely on eventual consistency. This means that different parts of the system may not immediately reflect the latest state, leading to temporary inconsistencies, but the data will get synced to all the services eventually and it will show the correct picture eventually. For example, in an e-commerce system, after an order is placed, the inventory might take a few seconds to update. We use techniques like distributed transactions, idempotent processing, and processing, and compensating actions to manage all this.
- **Ordering Guarantees:** Ensuring that events are processed in sequence is important. So typically what happens in a distributed system is that maintaining the correct order of events can be very challenging, especially when multiple consumers process events concurrently. For example, if a payment process event arrives before an order-placed event, the system may behave incorrectly. Solutions include partitioning the events by keys like user IDs and order IDs, or giving them sequence numbers, or implementing functionalities like event deduplication. But ordering of events is a challenge in case of event-driven architecture.
- **Fault Tolerance:** Failures can happen at any stage, even producers, brokers, or consumers. A resilient system should handle failures without losing events. For instance, if an email notification service goes down, events shouldn't be lost. They should be retried at a later stage. So there is something called dead letter queues, something called exponential back-off and retries and event reprocessing mechanisms, which will help us in ensuring reliability.

- **Debugging Complexity:** Since events move asynchronously across multiple services, debugging issues become harder than in traditional synchronous architecture. For example, if a customer support team needs to investigate a failed transaction, tracking the exact event flow can be very difficult. Distributed tracing tools like OpenTelemetry and all can be used to get some help here.

While these challenges exist with event-driven architecture, careful architectural decisions and the right tooling can make event-driven systems highly reliable, scalable, and efficient, even though it comes with some set of challenges.

Best Practices

To build a robust and reliable event-driven system, it's important to follow best practices. Let's go over some of the key recommendations to avoid some common pitfalls.

- **Use idempotent event processing to avoid duplicates.** Events may be delivered multiple times due to retries or network failures, and without proper handling, this can actually lead to duplicate processing. For example, if a payment process event is received twice, we don't want to charge the customer twice. To prevent this, event consumers should be idempotent, meaning they should be able to process the same event multiple times without any unintended side effects.
- **Implement dead-letter queues for failed messages.** Not all events will be processed successfully on the first attempt. So if a consumer continuously fails to process a message, it shouldn't get stuck in an infinite retry loop. For example, if an email notification service keeps failing due to an invalid email address, we need a way to capture and investigate this failure. You can solve this using dead-letter queues. Dead-letter queues provide a separate storage mechanism for failed messages, allowing engineers to inspect and manually handle them at a later point in time.
- **Choose the right event broker based on system needs.** We know that different event brokers serve different purposes. Choosing the right one depends on factors like message durability, ordering guarantees, and scalability. For example, Kafka is great for high-throughput event streaming, but RabbitMQ is better suited for traditional message queuing. So selecting the right event broker ensures the system remains efficient and meets performance expectations.
- **Ensure event versioning to handle schema changes.** Overtime event structures will evolve. Adding or removing fields in events can break the existing consumers if they are not handled correctly. For instance, if a user created an event initially containing name and email, but later added a phone number also, the older consumer might fail to process it. To handle this, we use event versioning strategies, such as adding new fields instead of modifying existing ones, or using schema registries like Apache Avro is one example.

So by following these best practices, we can build scalable, fault-tolerant, and maintainable event-driven architecture that supports our evolving business needs.

Use Cases of Event-Driven Architecture

Event-driven architecture is widely used across industries to enable real-time, scalable, and flexible systems. So let's try to explore some common use cases:

- **Logging & Auditing (Track Changes Overtime):** Keeping a record of a system's activity is crucial for debugging, compliance, and security. For example, every time a user updates their profile, an event can be logged in an audit trail. This helps organizations track changes and detect anomalies. Event-driven logging ensures that every action is captured without slowing down the main application.
- **Real-Time Notifications (Chat apps, stock price updates):** All our chat applications, stock price update applications use this. So there are many modern applications that require instant updates to keep users informed. For example, in a chat application, when a user sends a message, an event is triggered notifying the recipient in real-time.
- **Microservices Decoupling (Independent scalability & fault tolerance):** Microservices should be independently scalable and fault tolerant. One of the biggest advantages of event-driven architecture is that it enables loosely coupled microservices. For example, in an e-commerce system, the order service, payment service, and inventory service don't need to communicate directly. Instead, an order placed event can be triggered and payment processing will happen once this event is received by the payment service. This approach ensures flexibility and resilience and decoupled microservices.
- **IoT Systems (Sensor data processing):** IoT devices generate massive amounts of real-time data that needs to be processed efficiently. So if your smart home system is generating a lot of events and it wants to trigger actions like turning the air condition on or off, what we can do is that we can use event brokers like Kafka to handle this high frequency data efficiently.
- **E-commerce Order Processing:** This is actually a practical use case of e-commerce, where if the e-commerce site is well designed, what it does is it follows a sequence of events to ensure smooth order fulfillment. For example, when a customer places an order placed event is generated. It triggers payment processing, which then triggers inventory updates, which then triggers shipping notifications. All this happens asynchronously using events. This asynchronous event flow ensures scalability and prevents bottlenecks in high-traffic systems.

So these are some typical use cases that we have seen here to highlight the power of event-driven architecture. Event-driven architecture is very helpful in building responsive, scalable and resilient systems across various domains.

Summary & Key takeaways

- Event-driven architecture is key to building scalable, real-time, and decoupled systems.
- Choosing between Pub-Sub vs. Event Streaming depends on system needs.
- Architectural patterns like CQRS & Event Sourcing enhance flexibility.
- Challenges exist but can be mitigated with best practices.
- What's next:
 - Summary - Architecture patterns

Interview Questions on Event-Driven Architecture

- Fundamentals
 - What is Event-Driven Architecture (EDA), and how does it differ from traditional request-response architectures?
 - Explain the difference between Pub-Sub and Event Streaming models.
 - What are the key components of an event-driven system?
- Scalability & Fault Tolerance
 - What are some challenges of Event-Driven Architecture, and how do you handle eventual consistency?
 - How can you ensure event ordering in distributed event processing?
 - What are dead-letter queues (DLQs), and why are they important?
- Implementation & Technologies
 - What are the differences between Kafka, RabbitMQ, and AWS EventBridge?
 - How do you make event processing idempotent to avoid duplicate execution?
- Use Cases & Real-World Applications
 - Can you give a real-world example where Event-Driven Architecture is a better choice than a traditional architecture?
 - What strategies can you use to handle schema evolution in an event-driven system?